

Consistency Models cho ứng dụng mạng xã hội

Tình huống: Thiết kế một ứng dụng mạng xã hội đơn giản gồm:

- Đăng ký / Đăng nhập: xác thực username / password
- Cập nhật thông tin cá nhân (Profile): tên, ảnh đại diện
- Đăng bài viết (Post)
- Bình luận (Comment)
- Bộ đếm "Thích" (Like Count)

Yêu cầu: với **MOI** tính năng trên, hãy:

- Chọn mô hình nhất quán (Strong, Eventual, hoặc một biến thể cụ thể)
- Giải thích lý do lựa chọn (sử dụng danh đối)
- Đề xuất một công nghệ lưu trữ phù hợp

1. Đăng ký / Đăng nhập

Mô hình: Strong consistency (linearizability). Đăng ký linearizable trên username. Đăng nhập đọc từ primary; riêng đường **thu hồi** (đổi mật khẩu, khoá tài khoản, đăng xuất mọi thiết bị) bắt buộc strong.

Vì sao — đánh đổi. "Username là duy nhất" là **bất biến toàn cục** — loại bất biến duy nhất mà eventual consistency không giữ nổi. Hai replica cùng thấy `tam` chưa tồn tại, cùng chấp nhận, và hệ thống có hai tài khoản trùng tên. Không hoà giải tự động được: chọn người thắng nghĩa là **lấy tài khoản khỏi tay một người thật đã bắt đầu dùng nó**.

Đăng nhập là đường **đọc** nhưng vẫn không được eventual, vì ở đây dữ liệu cũ **không phải bất tiện mà là lỗi hỏng bảo mật**: đổi mật khẩu vì bị lộ mà replica còn hash cũ → mật khẩu cũ vẫn đăng nhập được. Tính bất đối xứng: **cấp quyền sai thì người dùng chịu thiệt, thu hồi trễ thì kẻ tấn công còn quyền truy cập**.

Cái giá: cả ghi lẫn đọc dồn về primary, nên **không đăng ký / đăng nhập được trong lúc failover**. Chấp nhận, vì lưu lượng xác thực nhỏ hơn lưu lượng đọc feed vài bậc — **strong consistency đắt theo lưu lượng, không đắt theo tính năng**.

Công nghệ: PostgreSQL, một primary. `users(id, username, password_hash, status)` với `UNIQUE (lower(username))` — để **ràng buộc của database** kiểm tra, không `SELECT` rồi `INSERT` (hai câu lệnh có khoảng hở cho hai request chen vào). Loại **Cassandra + LWT**: làm được, nhưng tài liệu nói `IF NOT EXISTS` *"should be used sparingly"* vì chạy Paxos ([nguồn](#)) — trả chi phí vận hành Cassandra để mua lại đúng thứ Cassandra cố tình từ bỏ.

2. Cập nhật Profile (tên, ảnh đại diện)

Mô hình: **Read-your-writes** + **Monotonic reads** (biến thể *session guarantees*) cho chính chủ, **Eventual** cho mọi người khác.

Vì sao — đánh đổi. Profile **không có bất biến toàn cục**: không ai thiệt hại nếu người B còn thấy ảnh cũ của A trong ba giây. Nên strong là trả giá vô ích — tên và ảnh xuất hiện trong **mọi** bài viết và bình luận trên màn hình, ép chúng đọc primary là tự tay giết primary.

Nhưng **chính chủ** thì nhận ra ngay: đổi ảnh, bấm F5, thấy ảnh cũ, kết luận là hỏng và **upload lại**. Dữ liệu không hề sai — chỉ là request rơi vào replica chưa kịp nhận.

Monotonic reads hay bị quên hơn và trông còn tệ hơn: nếu load balancer rải mỗi request sang một replica có độ trễ khác nhau, ảnh sẽ **nhảy qua nhảy lại** giữa cũ và mới theo từng lần F5. Eventual consistency **cho phép** điều đó, nhưng người dùng đọc nó là "hệ thống hỏng".

Cái giá: người khác thấy tên và ảnh cũ trong vài giây. Đổi lại, hai bảo đảm này rẻ đến bất ngờ vì **chỉ là định tuyến, không phải đồng thuận** — sticky một replica cho suốt phiên là có monotonic reads; sticky về primary N giây sau khi ghi là có read-your-writes.

Công nghệ: **PostgreSQL** (1 primary + N async replica) cho hàng `profiles`, và **object storage (S3 / MinIO) + CDN** cho file ảnh — không nhét ảnh vào `bytea` vì mỗi lần replicate là chép lại cả tấm ảnh. Đặt tên file theo **hash nội dung** `avatar/{userId}/{sha256}.webp`: ảnh mới là **URL mới** nên CDN không còn gì để invalidate, chỉ còn cột `avatar_url` phải nhất quán — mà nó đã nằm trong PostgreSQL và đã có read-your-writes.

3. Đăng bài viết (Post)

Mô hình: **Read-your-writes** cho tác giả, **Eventual** cho người theo dõi.

Vì sao — đánh đổi. Bài viết là một **hàng mới**, không phải sửa trạng thái dùng chung, nên không có bất biến nào để hai replica phá. Người theo dõi thấy bài chậm vài giây là điều mọi mạng xã hội đều chấp nhận — đó chính là thứ mua được độ trễ thấp và khả năng đọc từ replica.

Riêng tác giả thì phải thấy ngay, và lý do không phải thẩm mỹ: đăng xong không thấy bài mình → **đăng lại** → **bài trùng**, mà bài trùng thì người khác nhìn thấy. Đây là sự cố dữ liệu thật.

Cái giá, và chỗ dễ nhầm: "eventual" nói về *khi nào người khác thấy*, **không phải có được phép mất hay không**. Bài viết là nội dung người dùng tạo ra, mất là mất thật, không tính lại được — nên đọc eventual nhưng **ghi vẫn phải bền**, commit chờ ít nhất một bản sao. **Độ bền và độ nhất quán là hai trục khác nhau.**

Công nghệ: PostgreSQL — `posts(id, author_id, content, created_at)`, `index (author_id, created_at DESC)`, đọc từ async replica, và `UNIQUE (author_id, client_post_id)` để một lần retry vì mạng chập chờn không tạo ra bài thứ hai. **Cassandra** (partition key `author_id`, clustering `created_at DESC`) là lựa chọn đúng khi ghi vượt khả năng một primary — bài viết chỉ thêm, gần như không sửa, truy vấn luôn theo một tác giả. Nhưng chưa có vấn đề thì đừng trả giá: đổi sang Cassandra là mất `JOIN` và transaction đa bảng.

4. Bình luận (Comment)

Mô hình: Causal consistency, phạm vi là một bài viết.

Vì sao — đánh đổi. Đây là tính năng duy nhất mà eventual consistency làm **hỏng cách hiểu** của người đọc chứ không chỉ cho họ thấy số cũ. Hai kiểu hỏng:

- **Bình luận mờ côi** — bình luận tới replica trước cả bài viết mà nó thuộc về, màn hình có một bình luận trỏ vào một bài không tồn tại.
- **Hội thoại đọc ngược** — A viết "*ai đi không?*", B **đọc xong** mới trả lời "*tôi đi*". Trên replica nhận B trước A, thứ tự hiển thị thành "*tôi đi*" rồi mới "*ai đi không?*". Ghi của B **nhân quả sau** ghi của A, và eventual consistency **không hứa gì** về thứ tự đó.

Không lấy luôn Strong vì phải trả chi phí đồng thuận cho **mọi** bình luận, mà ta **không cần** thứ tự toàn cục — chỉ cần thứ tự **bên trong một bài** là đúng. Không ai mở hai bài cạnh nhau rồi so thứ tự bình luận chéo giữa chúng.

Cái giá: thứ tự chỉ bảo đảm trong phạm vi một bài; giữa hai bài khác nhau thì không hứa gì.

Công nghệ: Cassandra, PRIMARY KEY ((post_id), created_at, comment_id), đọc/ghi LOCAL_QUORUM. Chọn nó vì post_id làm **partition key** nên mọi bình luận của một bài nằm trên cùng một tập replica — bài toán "thứ tự nhân quả toàn cục" co lại thành "thứ tự trong một partition", vốn có sẵn. Lưu thêm parent_comment_id để dựng lại cây; **đừng sắp xếp bằng đồng hồ client**, lệch vài trăm ms là câu trả lời mang timestamp trước câu hỏi. Last-write-wins của Cassandra vô hại ở đây vì mỗi bình luận là **một hàng riêng** — không có ô nào để tranh chấp.

5. Bộ đếm "Thích" (Like Count)

Mô hình: Eventual consistency cho con số hiển thị.

Vì sao — đánh đổi. Chi phí của một lần đọc cũ ở đây **đúng bằng không**: không có bất biến, không ai mất tiền, và **không ai — kể cả tác giả — phân biệt được 10.412 với 10.415**. Đổi lại là thứ đáng giá nhất: luồng ghi lớn nhất của hệ không phải chạm tới bất kỳ sự đồng thuận nào.

Riêng "like của mình phải đỏ lên ngay" là **yêu cầu giao diện, không phải yêu cầu nhất quán** — xử lý bằng optimistic update ở client, không cần đổi mô hình lưu trữ.

Cái giá, và cách trả nó: bộ đếm **không idempotent** — một lần tăng bị timeout rồi retry có thể cộng hai lần. Tài liệu Cassandra nói thẳng "*Counter updates are, by nature, not idempotent*", và replay "*may or may not lead to an over count*" ([nguồn](#)); Redis `INCR` y hệt. Nên **không được để bộ đếm làm nguồn sự thật**: nguồn sự thật là tập `likes(post_id, user_id)` — idempotent sẵn nhờ khoá chính, bấm năm lần vẫn một hàng — còn con số chỉ là **giá trị dẫn xuất, luôn đếm lại được**. Nhờ vậy mọi sai lệch đều tự sửa được.

Công nghệ: Cassandra `PRIMARY KEY ((post_id), user_id)` cho tập like — câu hỏi nóng nhất "*tôi đã thích bài này chưa*" thành một lần tra khoá chính trong đúng một partition — cộng Redis `HINCRBY` cho con số hiển thị, coi là **cache được phép mất**, kèm job đối soát định kỳ đếm lại từ tập like. Loại `UPDATE posts SET like_count = like_count + 1` của PostgreSQL: chính xác tuyệt đối, nhưng mọi like của một bài viral đánh vào **cùng một hàng** và xếp hàng sau một khoá.